

Web technológiák 2

Hardver Nyilvántartó Rendszer

Hallgató: Dobozi Botond

Neptun-kód: HYS4P5

Tárgy: Web technológiák 2

Félév: 2025/26 2. félév

Projektjegyzőkönyv

2026. május 11.

Tartalomjegyzék

1. Bevezetés	2
2. Technológiai Stack	3
2.1. Backend: Node.js és Express.js	3
2.2. Frontend: Next.js (App Router)	3
2.3. Stílus: Tailwind CSS	3
3. Backend Architektúra	4
3.1. Adatmodellek (Models)	4
3.2. Szolgáltatások és Kontrollerek	4
3.3. Adatbázis konfiguráció és Seedelés	4
4. Frontend Felépítés és Navigáció	5
4.1. Oldalstruktúra (App Router)	5
4.2. Komponensrendszer	5
5. Globális Állapotkezelés	6
5.1. Context és Reducer	6
5.2. Actions	6
6. Implementációs Részletek	7
6.1. CRUD Funkcionalitás	7
6.2. Hibatűrés és Típusbiztonság	7
7. Üzemeltetés és Telepítés	8
8. Összegzés	9

1. fejezet

Bevezetés

A modern informatikai környezetekben az eszközpark nyomon követése kritikus feladat. Jelen projekt egy olyan komplex, full-stack megoldást kínál, amely lehetővé teszi a hardvereszközök központi adminisztrációját, statisztikai elemzését és digitális leltározását.

A fejlesztés során az elsődleges szempont a robusztusság és a típusbiztonság volt, ezért a teljes stack (frontend és backend egyaránt) **TypeScript** alapokon nyugszik. Ez a megközelítés minimalizálja az adatstruktúrák közötti inkonzisztenciát és modern fejlesztői élményt biztosít.

2. fejezet

Technológiai Stack

A projekt architektúrája szétválasztott (decoupled) felépítésű, ahol a kliens és a szerver független modulokként funkcionálnak.

2.1. Backend: Node.js és Express.js

A szerveroldali réteg felelős az üzleti logika kiszolgálásáért és az adatbázis műveletekért. Az Express.js keretrendszert TypeScript-tel kiegészítve egy szigorúan típusos API-t hoztam létre. Az adatok perzisztenciáját egy **MongoDB** NoSQL adatbázis biztosítja a Mongoose Object Modeling segítségével.

2.2. Frontend: Next.js (App Router)

A felhasználói felület a legfrissebb Next.js keretrendszert használja. Az `app` router struktúra lehetővé tette az útvonal-alapú kód-szétválasztást és a szerveroldali renderelés (SSR) előnyeinek kihasználását a statisztikai adatok gyors betöltéséhez.

2.3. Stílus: Tailwind CSS

A vizuális megjelenésért a Tailwind CSS felel, amely biztosítja a reszponzivitást és a konzisztens megjelenést a különböző hardver-adatlapok és adminisztrációs táblázatok között.

3. fejezet

Backend Architektúra

A backend kódstruktúrája a `backend/src` mappában rétegelt felépítést követ.

3.1. Adatmodellek (Models)

A `product.model.ts` fájl definiálja a hardverek sémáját. Bár a fájlnev "product", a logikai tartalom hardver-specifikus attribútumokat tartalmaz (típus, specifikációk, sorozatszám, ár, készlet). Az üzeneteket a `message.model.ts` kezeli.

3.2. Szolgáltatások és Kontrollerek

A `services/` réteg tartalmazza az adatbázis-manipulációs logikát, míg a `controllers/` réteg felel a beérkező HTTP kérések (request) és válaszok (response) kezeléséért.

3.3. Adatbázis konfiguráció és Seedelés

A `db.ts` végzi a csatlakozást, a `seed.ts` pedig felelős a rendszer kezdeti feltöltéséért (pl. alapértelmezett processzorok, memóriák bevitele teszteléshez).

4. fejezet

Frontend Felépítés és Navigáció

A frontend struktúrája közvetlenül tükrözi az alkalmazás funkcióit.

4.1. Oldalstruktúra (App Router)

- **Nyilvántartás:** Az összes regisztrált hardver listázása.
- **Statisztikák:** A készlet értékének és eloszlásának vizuális elemzése.
- **Admin:** Az eszközök hozzáadása, módosítása és törlése (CRUD).
- **Kapcsolat:** Hibajelentések vagy beszerzési igények leadása.

4.2. Komponensrendszer

A `components/` mappában található elemek (pl. `StatCard.tsx`, `ProductCard.tsx`) biztosítják a modularitást. A statisztikai kártyák kiemelt szerepet kapnak a vezetői összefoglalók megjelenítésében.

5. fejezet

Globális Állapotkezelés

A projekt egyik technikai különlegessége a React **Context API** és a **useReducer** hook kombinációja.

5.1. Context és Reducer

A `context/` mappában található `GlobalContext.tsx` és `Reducer.tsx` fájlok kezelik az alkalmazás globális állapotát. Ez teszi lehetővé, hogy a backendről lekért hardverlista azonnal frissüljön az admin felületen történő módosítás után, anélkül, hogy a teljes oldalt újra kellene tölteni.

5.2. Actions

Az `Actions.ts` definiálja a végrehajtható műveleteket (pl. `FETCH_HARDWARE`, `DELETE_ITEM`), biztosítva a prediktív adatkezelést.

6. fejezet

Implementációs Részletek

6.1. CRUD Funkcionalitás

A rendszer lehetővé teszi új hardverek felvitelét, meglévők adatainak (pl. órajel, kapacitás) frissítését, valamint a selejtezett eszközök törlését. Minden művelet aszinkron módon, a `services/` rétegen keresztül kommunikál a REST API-val.

6.2. Hibatűrés és Típusbiztonság

A TypeScript interfészek központi helyen, a `types/` mappában vannak deklarálva, így a frontend pontosan tudja, milyen formátumú adat érkezik a szervertől.

7. fejezet

Üzemeltetés és Telepítés

A projekt elindításához szükséges lépések:

1. **Backend:** `npm install` után `npm run dev`. A szerver alapértelmezetten a 5000-es porton fut.
2. **Frontend:** `npm install` után `npm run dev`. A felület a 3000-es porton érhető el.
3. **Környezeti változók:** A `.env` fájlban kell megadni a MongoDB elérési útvonalát.

8. fejezet

Összegzés

Az elkészült Hardver Nyilvántartó Rendszer bizonyítja a modern webes technológiák hatékonyságát az adminisztratív folyamatokban. A Next.js és az Express.js párosítása egy rendkívül gyors, típusbiztos és könnyen bővíthető rendszert eredményezett, amely alkalmas nagyobb eszközparkok kezelésére.